BÀI THỰC HÀNH CÔNG NGHỆ .NET

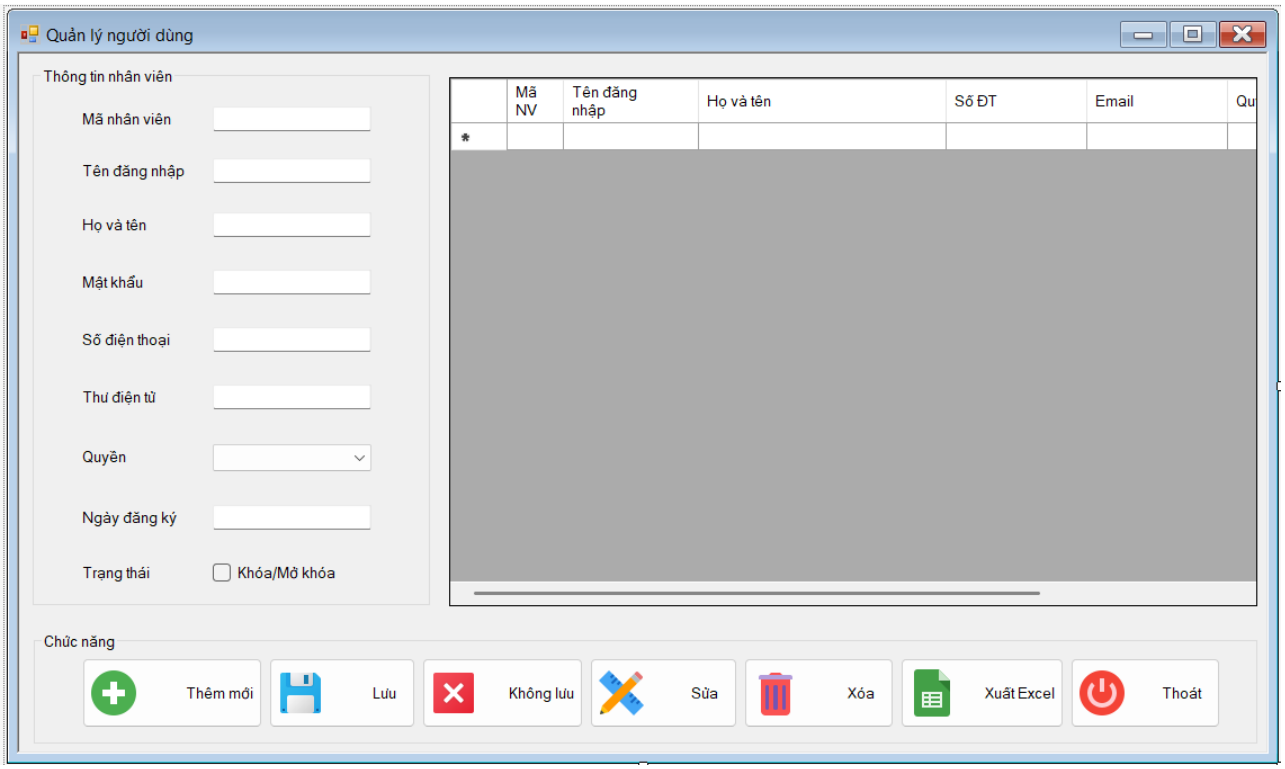
(ĐỒ ÁN: XÂY DỰNG PHẦN MỀM QUẢN LÝ CHO THUÊ SÁCH)

BÀI THỰC HÀNH #3: LẬP TRÌNH HIỂN THỊ/THÊM/SỬA/XÓA TRÊN GIAO DIỆN QUẢN LÝ NGƯỜI DÙNG SỬ DỤNG CÔNG NGHỆ LINQ

1. Nội dung chuẩn bị

- Các tệp cơ sở dữ liệu: Bài TH01 (bảng **tblUsers**)

- Giao diện **frmUsers**: Bài TH02



2. Lập trình Hiển thị/Thêm/Sửa/Xóa sử dụng LINQ (Language Integrated Query -Truy vấn tích hợp ngôn ngữ).

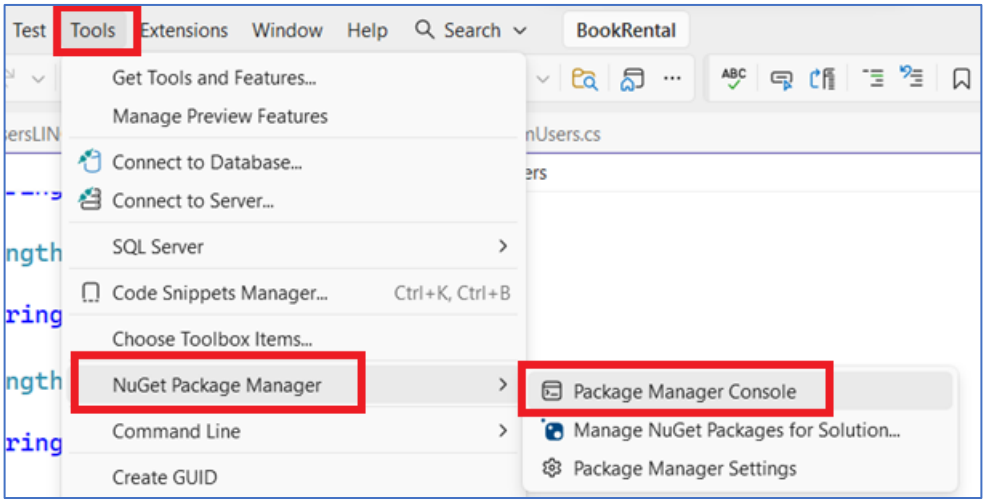
- Mục đích:

  + Hiển thị thông tin trong bảng tblUsers lên dgvUser

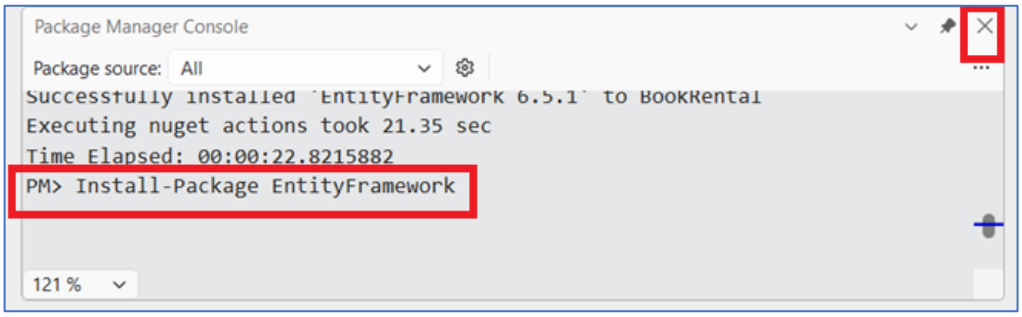
  + Thực hiện việc Thêm/Sửa/Xóa thông tin trong bảng tblUsers

- Cài đặt gói thư viện **EntityFramework** (chỉ làm 1 lần cho toàn bộ project)

  + Vào **Tools**, Chọn **NuGet Package Manager**, chọn **Package Manager Console**



- Gõ lệnh **Install-Package EntityFramework** trong khung PM>



- Thực hiện xong đóng khung.

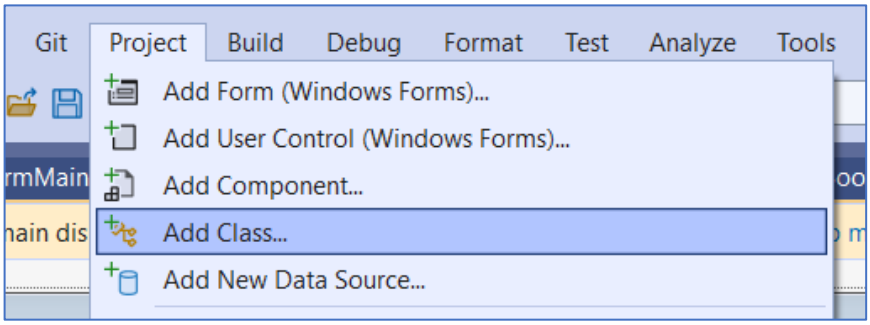
2.1. Tạo lớp tblUser.cs

- Mục đích: Lớp này chứa các thuộc tính để lưu trữ dữ liệu trong bảng tblUsers (trong bài thực hành TH01)

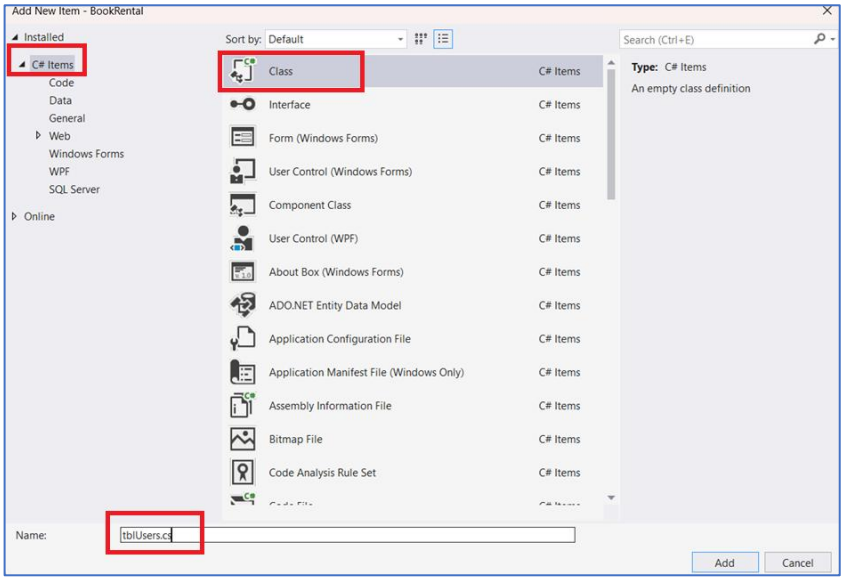
- Thực hiện:

+ Vào **Project**, Chọn **Add Class…**

(Xuất hiện menu như sau)



+ Gõ **tblUsers.cs** trong hộp **Name**



+ Xuất hiện giao diện sau

```
6
7   namespace BookRental
8   {
        0 references
9       internal class tblUsers
10      {
11      }
12  }
```

+ Gõ nội dung sau vào trong tệp **tblUsers.cs**

```
1   using System;
2   using System.ComponentModel.DataAnnotations;
3   using System.ComponentModel.DataAnnotations.Schema;
4
5   namespace BookRental
6   {
7       [Table("tblUsers")]
        4 references
8       internal class tblUsers
9       {
10          [Key] // Khai báo Khóa chính
11          [DatabaseGenerated(DatabaseGeneratedOption.Identity)] // Tự động tăng
            2 references
12          public int UserID { get; set; }
13
14          [Required]
15          [StringLength(50)]
            2 references
16          public string Username { get; set; }
17
18          [Required]
19          [StringLength(255)]
            1 reference
20          public string Password { get; set; }
21
22          [StringLength(100)]
            2 references
23          public string FullName { get; set; }
24
25          [StringLength(20)]
            2 references
26          public string Phone { get; set; }
27
28          [StringLength(100)]
            2 references
29          public string Email { get; set; }
30
31          [StringLength(20)]
            2 references
32          public string Role { get; set; }
33
            2 references
34          public bool Status { get; set; }
35
            2 references
36          public DateTime CreatedDate { get; set; }
37      }
38  }
```

2.2. Cấu hình Chuỗi kết nối (App.config)

- Mở tệp App.config, thêm các dòng sau (màu đỏ) vào sau thẻ **</configSections>**, có thể copy nội dung ở dưới rồi chỉnh sửa tên phù hợp (tương tự như làm ADO.NET ở bài thực hành trước)

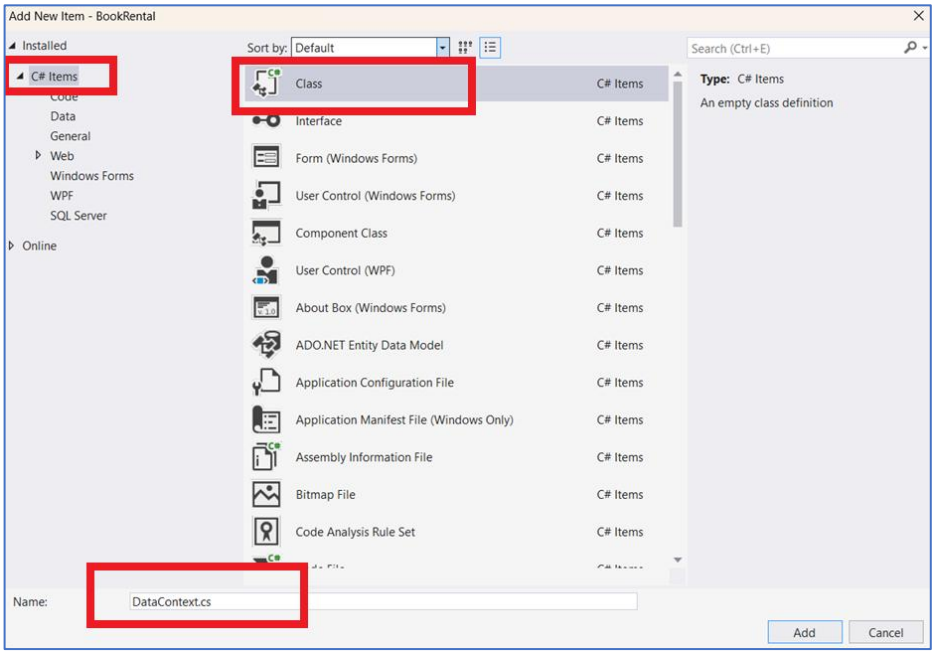
```
Program.cs   App.config   DataContext.cs   tblUsers.cs*   frmUsersLINQ.cs [Design]   frmUsersLINQ.cs   frmUsers.cs
<?xml version="1.0" encoding="utf-8"?>
<configuration>

  <configSections>
    <!-- For more information on Entity Framework configuration, visit http://go.microsoft.com/fwlink/?LinkID=237468 -->
    <section name="entityFramework" type="System.Data.Entity.Internal.ConfigFile.EntityFrameworkSection, EntityFramework, Version=6.0.0.0, Cu
  </configSections>
  <connectionStrings>
    <add name="MyConnectionString"
         connectionString="Data Source=SONCT\SQL2019;Initial Catalog=BOOKRENTAL;Integrated Security=True"
         providerName="System.Data.SqlClient" />
  </connectionStrings>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.7.2" />
  </startup>
  <entityFramework>
    <providers>
      <provider invariantName="System.Data.SqlClient" type="System.Data.Entity.SqlServer.SqlProviderServices, EntityFramework.SqlServer" />
    </providers>
  </entityFramework>
</configuration>
```

```
<connectionStrings>
        <add name="MyConnectionString"
         connectionString="Data Source=SONCT\SQL2019;Initial
Catalog=BOOKRENTAL;Integrated Security=True"
         providerName="System.Data.SqlClient" />
</connectionStrings>
```

2.3. Tạo lớp **DataContext.cs** (chỉ tạo 1 lần và dùng chung sau này)

- Vào Project, chọn Add Class, gõ **DataContext.cs**

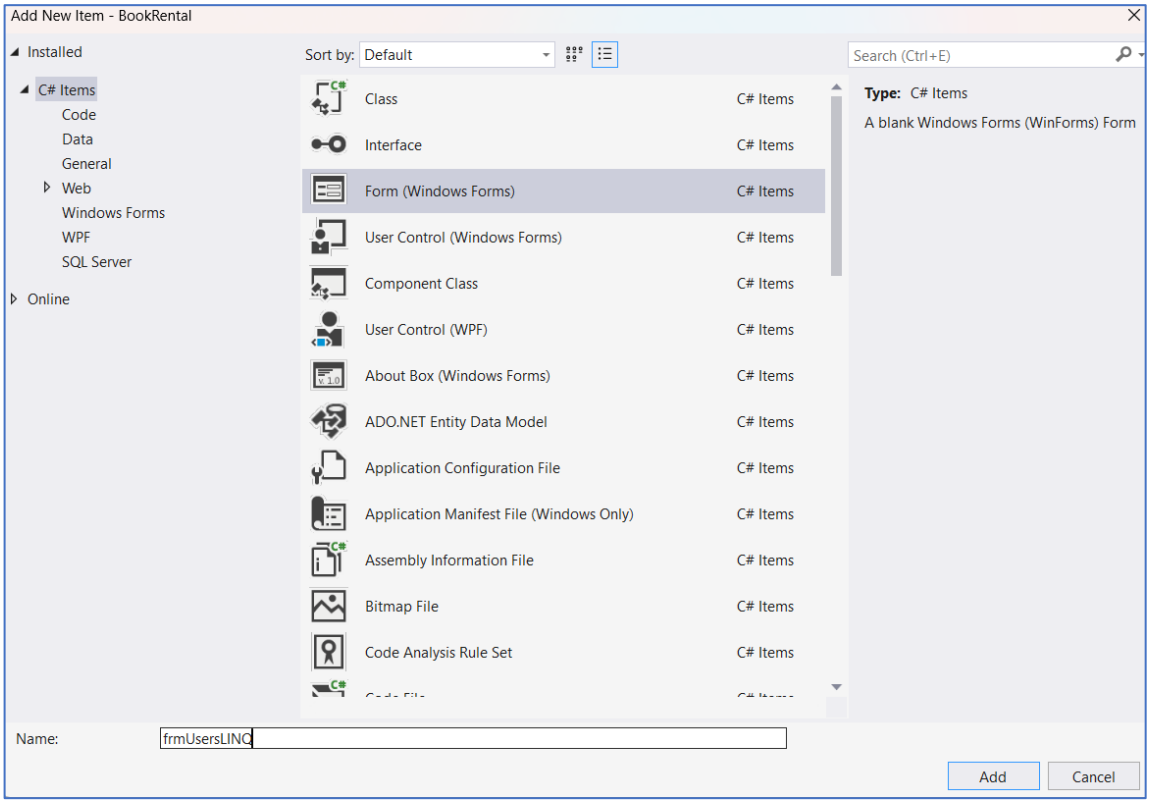


- Gõ nội dung sau vào tệp **DataContext.cs**

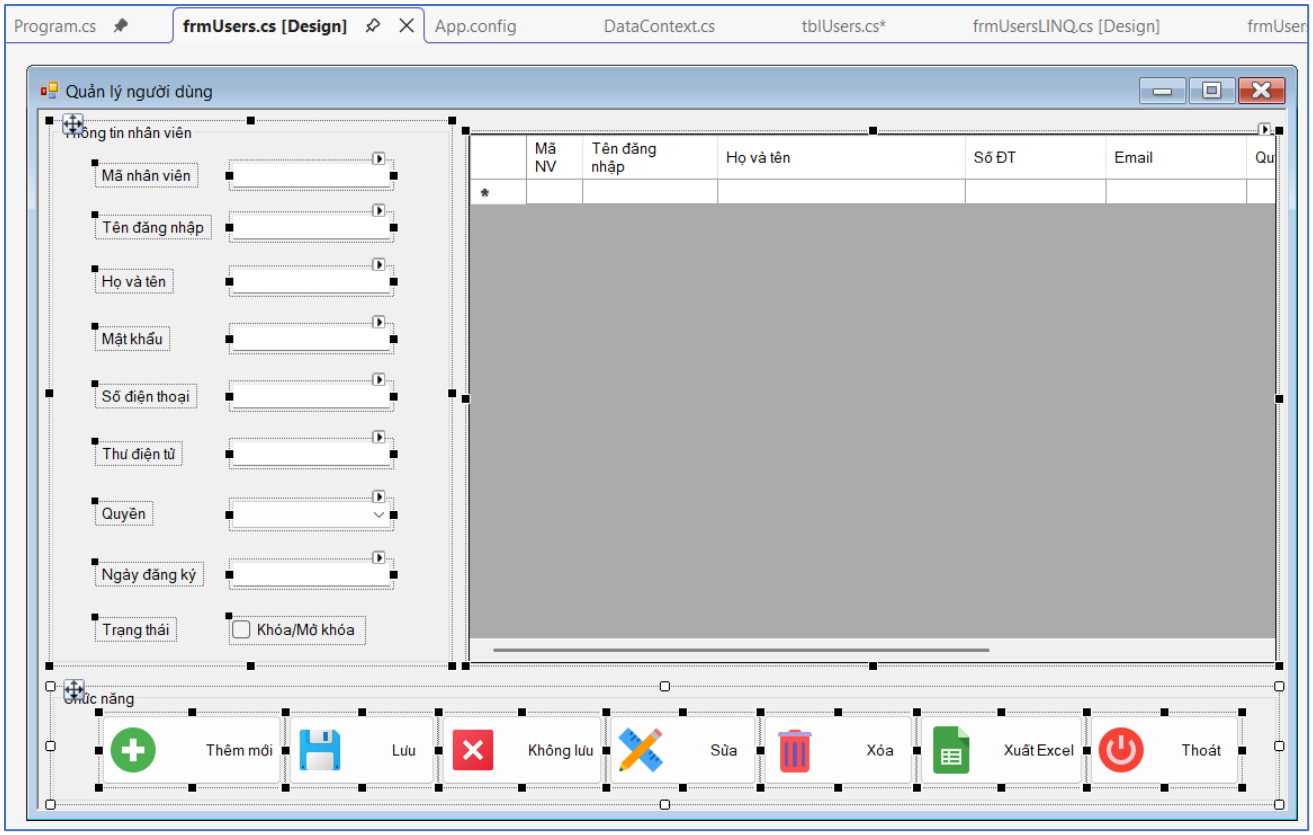
```
1   using System;
2   using System.Collections.Generic;
3   using System.Data.Entity;
4   using System.Linq;
5   using System.Text;
6   using System.Threading.Tasks;
7
8   namespace BookRental
9   {
        3 references
10      internal class DataContext : DbContext
11      {
            1 reference
12          public DataContext() : base("name=MyConnectionString")
13          {
14          }
            6 references
15          public DbSet<tblUsers> Users { get; set; }
16      }
17  }
```

2.5. Tạo form mới frmUsersLINQ như sau

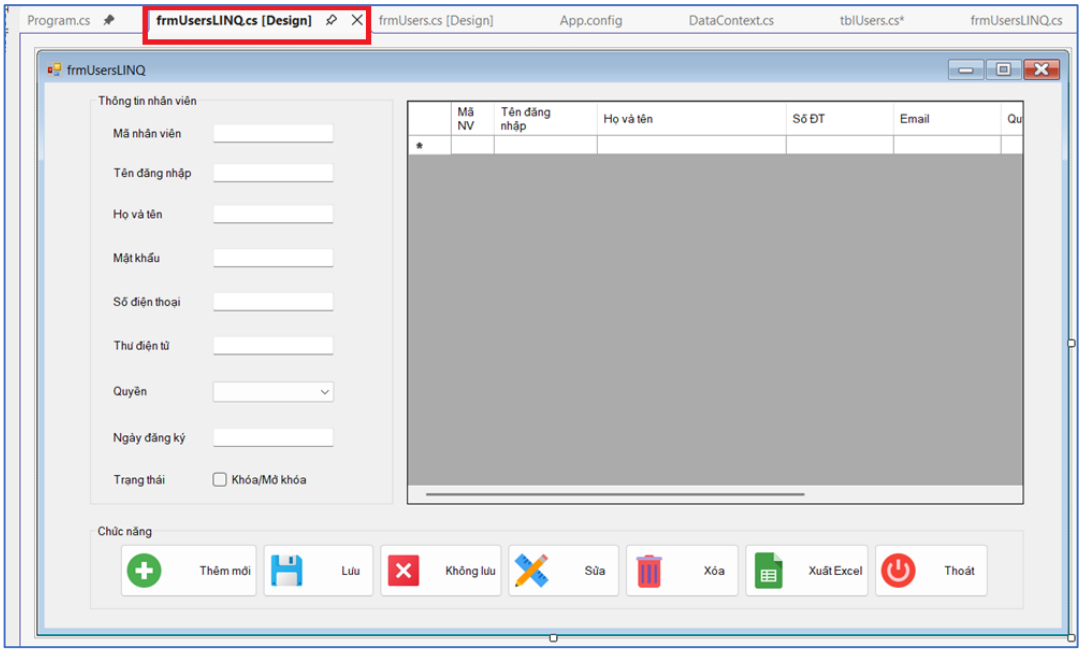
- Vào Project, Add Form, gõ frmUsersLINQ như sau



- Mở form frmUsers đã thực hiện trước đó ở chế độ Design, ấn CTRL+A để chọn toàn bộ các điều khiển, trên form này, ấn CTRL + C để copy (nếu chưa làm frmUsers thì xem lại bài thực hành TH02)



- Mở lại form frmUsersLINQ, ấn CTRL + V để dán toàn bộ giao diện vào form mới (đỡ mất công thiết kế lại), căn chỉnh phù hợp sau khi copy, ta có như sau



2.6. Lập trình Hiển thị/Thêm/Sửa/Xóa trên form frmUsersLINQ

- Nhấp đúp vào thanh tiêu đề của from sẽ xuất hiện phần code, gõ các nội dung sau

```
using System;
using System.Data;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Windows.Forms;

namespace BookRental
{
    3 references
    public partial class frmUsersLINQ : Form
    {
        DataContext db = new DataContext();
        bool AddNew = false;
        1 reference
        public frmUsersLINQ()
        {
            InitializeComponent();
        }

        4 references
        private void setContol(bool check)
        {
            txtUserID.Enabled = false;
            txtUsername.Enabled = check;
            txtFullName.Enabled = check;
            txtPassword.Enabled = check;
            txtPhone.Enabled = check;
            txtEmail.Enabled = check;
            txtCreatedDate.Enabled = false;
            cbbRole.Enabled = check;
            ckbStatus.Enabled = check;
            btnSave.Enabled = check;
            btnCancel.Enabled = check;
            btnAdd.Enabled = !check;
            btnEdit.Enabled = !check;
            btnDelete.Enabled = !check;
            btnExcel.Enabled = !check;
            btnExit.Enabled = !check;
            dgvUsers.Enabled = !check;
        }
        4 references
        private void LoadGridData()
        {
            // Lấy toàn bộ danh sách từ bảng tblUsers bằng LINQ
            var data = from u in db.Users
                       select u;
            dgvUsers.DataSource = data.ToList();
            setContol(false);
        }
        1 reference
        private void frmUsersLINQ_Load(object sender, EventArgs e)
        {
            dgvUsers.AutoGenerateColumns = false;
            dgvUsers.AllowUserToAddRows = false;
            LoadGridData();
        }
    }
}
```

- Trong đó phương thức **setControl(…)**, **frmUsersLINQ_Load(…)** tương tự bài TH03

- Lập trình nút Xóa (tương tự TH3), nhấp đúp vào nút Xóa, gõ lệnh

```
private void btnExit_Click(object sender, EventArgs e)
{
    if (MessageBox.Show("Bạn muốn thoát không?", "Thông báo",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question) == DialogResult.Yes)
    {
        this.Close();
    }
}
```

- Lập trình sự kiện **dgvUsers_CellEnter(object sender, DataGridViewCellEventArgs e)** tương tự bài TH03 (có thể copy nội dung từ bên bài TH03 sang)

+ Nhấp phải chuột vào dgvUsers, chọn sự kiện, nhấp đúp vào CellEnter (chi tiết tại TH03)

```
private void dgvUsers_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    int i = e.RowIndex;
    if (i < 0) return;
    txtUserID.Text = dgvUsers.Rows[i].Cells["UserID"].Value.ToString();
    txtUsername.Text = dgvUsers.Rows[i].Cells["UserName"].Value.ToString();
    txtFullName.Text = dgvUsers.Rows[i].Cells["FullName"].Value.ToString();
    txtPassword.Text = dgvUsers.Rows[i].Cells["Password"].Value.ToString();
    txtPhone.Text = dgvUsers.Rows[i].Cells["Phone"].Value.ToString();
    txtEmail.Text = dgvUsers.Rows[i].Cells["Email"].Value.ToString();
    cbbRole.Text = dgvUsers.Rows[i].Cells["Role"].Value.ToString();
    txtCreatedDate.Text = Convert.ToDateTime(dgvUsers.Rows[i].Cells["CreatedDate"].Value).ToString("dd/MM/yyyy");
    if ((bool)dgvUsers.Rows[i].Cells["Status"].Value == true)
    {
        ckbStatus.Checked = true;
        ckbStatus.Text = "Tài khoản đang mở";
    }
    else
    {
        ckbStatus.Checked = false;
        ckbStatus.Text = "Tài khoản đã đóng";
    }
}
```

- Lập trình nút xóa, nhấp đúp vào nút Xóa trên giao diện, gõ nội dung như sau

```
1 reference
private void btnDelete_Click(object sender, EventArgs e)
{
    if (dgvUsers.CurrentRow == null) return;
    if (MessageBox.Show("Bạn muốn xóa bản ghi này không?", "Thông báo",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Stop) == DialogResult.Yes)
    {
        int id = (int)dgvUsers.CurrentRow.Cells["UserID"].Value;
        var userDelete = db.Users.SingleOrDefault(u => u.UserID == id);

        if (userDelete != null)
        {
            db.Users.Remove(userDelete);
            db.SaveChanges();
            LoadGridData();
        }
    }
}
```

- Lập trình nút Thêm/Sửa/Không lưu (tương tự TH03, có thể copy), nhấp đúp nút tương ứng

+ Nút Thêm mới

```
1 reference
private void btnAdd_Click(object sender, EventArgs e)
{
    AddNew = true;
    setContol(true);
    txtUserID.Clear();
    txtUsername.Clear();
    txtFullName.Clear();
    txtPassword.Clear();
    txtPhone.Clear();
    txtEmail.Clear();
    cbbRole.Text = "Staff";
    txtCreatedDate.Text = DateTime.Now.ToString("dd/MM/yyyy");
    ckbStatus.Checked = true;
    txtUsername.Focus();
}
```

+ Nút Sửa

```
1 reference
private void btnEdit_Click(object sender, EventArgs e)
{
    AddNew = false;
    setContol(true);
    txtUsername.Enabled = false;
    txtUsername.Focus();
}
```

+ Nút Không lưu

```
1 reference
private void btnCancel_Click(object sender, EventArgs e)
{
    setContol(false);
    //Lấy lại thông tin từ lưới lên các TextBox
    if (dgvUsers.CurrentRow != null)
    {
        int rowIndex = dgvUsers.CurrentRow.Index;
        int colIndex = dgvUsers.CurrentCell.ColumnIndex;
        DataGridViewCellEventArgs args = new DataGridViewCellEventArgs(colIndex, rowIndex);
        dgvUsers_CellEnter(dgvUsers, args);
    }
}
```

- Thêm phương thức **GetMD5(….)** (tương tự TH03, có thể copy)

```
1 reference
public string GetMD5(string password)
{
    using (MD5 md5 = MD5.Create())
    {
        byte[] inputBytes = Encoding.UTF8.GetBytes(password);
        byte[] hashBytes = md5.ComputeHash(inputBytes);
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < hashBytes.Length; i++)
        {
            sb.Append(hashBytes[i].ToString("x2"));
        }
        return sb.ToString();
    }
}
```

- Lập trình nút Lưu, có kiểm tra trùng dữ liệu (Username)

```
159         private void btnSave_Click(object sender, EventArgs e)
160         {
161             if (txtUsername.Text.Trim() == "")
162             {
163                 MessageBox.Show("Thông tin tên sử dụng không được để trống!", "Thông báo");
164                 txtUsername.Focus();
165                 return;
166             }
167             //Tương tự kiểm tra các thông tin khác
168
169             if (AddNew) //Nếu trước đó ấn vào nút thêm mới thì đoạn này sẽ thực hiện
170             {
171                 //Kiểm tra trùng Username
172                 string inputUsername = txtUsername.Text.Trim();
173                 bool isExisted = db.Users.Any(u => u.Username == inputUsername);
174                 if (isExisted)
175                 {
176                     MessageBox.Show("Tên đăng nhập này đã tồn tại! Vui lòng chọn tên khác.",
177                                     "Thông báo", MessageBoxButtons.OK, MessageBoxIcon.Warning);
178                     txtUsername.Focus();
179                     return;
180                 }
181                 //Nếu không trùng thì tiến hành thêm mới
182                 tblUsers newUser = new tblUsers
183                 {
184                     Username = txtUsername.Text.Trim(),
185                     Password =  GetMD5(txtPassword.Text.Trim()),
186                     FullName = txtFullName.Text.Trim(),
187                     Phone = txtPhone.Text.Trim(),
188                     Email = txtEmail.Text.Trim(),
189                     Role = cbbRole.Text.Trim(),
190                     Status = ckbStatus.Checked,
191                     CreatedDate = DateTime.Now
192                 };
193
194                 db.Users.Add(newUser);
195                 db.SaveChanges();
196                 LoadGridData();
197             }
198             else //Nếu trước đó ấn vào nút sửa thì đoạn này sẽ thực hiện
199             {
200                 if (dgvUsers.CurrentRow == null) return;
201
202                 int id = int.Parse(txtUserID.Text);
203                 // Tìm đối tượng cần sửa bằng LINQ
204                 tblUsers userUpdate = db.Users.SingleOrDefault(u => u.UserID == id);
205
206                 if (userUpdate != null)
207                 {
208                     userUpdate.FullName = txtFullName.Text.Trim();
209                     userUpdate.Phone = txtPhone.Text.Trim();
210                     userUpdate.Email = txtEmail.Text.Trim();
211                     userUpdate.Role = cbbRole.Text.Trim();
212                     userUpdate.Status = ckbStatus.Checked;
213                     userUpdate.CreatedDate = DateTime.ParseExact(txtCreatedDate.Text, "dd/MM/yyyy", null);
214
215                     db.SaveChanges();
216                     LoadGridData();
217                 }
218             }
219         }
```

Lưu ý:

- Sau này khi làm việc với các form khác thì bước 2.2 không cần thực hiện
- Tạo một lớp tương ứng như bước 2.1
- Thêm một dòng tương ứng trong bước 2.3

Ví dụ:

- Tạo form để kết nối với bảng tblCategories thì tạo lớp **tblCategories.cs** tương ứng
- Trong DataContext.cs thêm dòng:

  public DbSet<tblCategories> Categories { get; set; }
- Xem hình

```
internal class DataContext : DbContext
{
    1 reference
    public DataContext() : base("name=MyConnectionString")
    {
    }
    6 references
    public DbSet<tblUsers> Users { get; set; }
    0 references
    public DbSet<tblCategories> Categories { get; set; }

}
```